Revisão Prática para Avaliação

1. Fluxo de uma aplicação Fullstack (Lista de Compras)

FrontEnd (HTML/JS) → coleta os dados do usuário.

Requisição HTTP (geralmente POST) → enviada para o Express.

Backend (Express + Mongoose) → valida e envia os dados para o MongoDB.

MongoDB (via Mongoose) → persiste os dados.

Resposta → retorna ao FrontEnd para exibir a atualização da lista.

Exercício prático:
Implemente um fetch no script.js para enviar um item via POST:

2. ORM/ODM – Papel na aplicação

ORM (Object Relational Mapping) → usado com bancos relacionais (Postgres, MySQL).

ODM (Object Document Mapping) → usado com MongoDB (Mongoose).

Função principal: mapear tabelas/coleções em objetos da aplicação.

Exercício prático:
Mostre como ficaria um modelo em Prisma (ORM) e um em Mongoose (ODM) para uma tabela/coleção de tasks.

3. Métodos HTTP – Atualização de recursos

POST → criação.

GET → leitura.

PUT → atualização completa.

PATCH → atualização parcial.

DELETE → exclusão.

Exercício prático:
Implemente rota em Express para atualizar uma tarefa inteira:

4. Mongoose – Campos obrigatórios e únicos

Para garantir unicidade e obrigatoriedade em um campo email:

```
const UserSchema = new mongoose.Schema({
email: { type: String, required: true, unique: true }
});
```

Exercício prático:

Tente cadastrar dois usuários com o mesmo email → o Mongoose deve lançar erro.

5. Prisma ORM x Mongoose ODM

Prisma → ORM para bancos relacionais (Postgres, MySQL, SQLite, etc.)
Mongoose → ODM para MongoDB (NoSQL).
Ambos mapeiam dados → objetos, mas cada um é especializado em um tipo de banco.

Exercício prático:
Monte um mini-quadro comparativo no caderno:

```
Quadro comparativo:

| Ferramenta      |     Tipo  |     Usado em    |     Exemplo |
|-----------------------------------------------------------|
| Prisma          | ORM       |     SQL         | PostgreSQL      |
| Mongoose        | ODM       | NoSQL           | MongoDB         |
```

■ Questões para Treino

6. Em uma aplicação REST, qual método HTTP deve ser utilizado para deletar permanentemente um registro no banco de dados?

7. No uso do Mongoose, qual comando deve ser usado para buscar todos os documentos da coleção 'tasks'?

8. Um desenvolvedor precisa atualizar apenas o campo 'completed' de uma tarefa em uma API REST. Qual método é mais adequado semanticamente?

9. Em uma aplicação Fullstack com Express e MongoDB, qual é a função do Mongoose?

10. Durante o desenvolvimento de um sistema de controle de estoque com Node.js, os alunos precisaram implementar a exclusão de produtos pela API REST. Qual método HTTP deve ser utilizado para representar semanticamente essa operação?

11. Um professor pediu que os alunos implementassem uma rota em Express que retornasse todos os usuários cadastrados no MongoDB. Qual alternativa mostra corretamente o tipo de requisição HTTP que deve ser usado?

12. Em uma aplicação que utiliza Mongoose, um campo senha precisa ser obrigatório e armazenado como texto. Qual seria a definição correta no schema?

13. Um grupo de alunos está implementando uma aplicação com Prisma ORM. Eles precisam criar um modelo Produto com id numérico e auto incremento, nome obrigatório e preço do tipo decimal. Qual alternativa representa corretamente esse modelo em Prisma?

a)
```
model Produto {
  id Int
  nome String
  preco Int
}
```

b)
```
model Produto {
  id Int @id @default(autoincrement())
  nome String
  preco Decimal
}
```

c)

```
model Produto {
  id String
  nome String
  preco Float
}
```

d)

```
model Produto {
  id Int @default(autoincrement())
  nome Int
  preco String
}
```

e)

```
model Produto {
  id Serial
  nome Text
  preco Double
}
```

14. Em uma API RESTful para gerenciamento de alunos, qual método deve ser utilizado quando queremos alterar apenas o campo “telefone” de um aluno já cadastrado?

15. Ao analisar o arquivo schema.prisma abaixo, qual será o nome da coleção criada no MongoDB?

```
model ShoppingItem {
  id    String @id @default(auto()) @map("_id") @db.ObjectId
  name  String
}
```

A) _id
B) ShoppingItem
C) ObjectId
D) shoppingitems
E) model

16. Qual comando do terminal é utilizado para abrir a interface visual do Prisma e gerenciar os dados do banco sem digitar código?

A) npx prisma generate

B) npx prisma db push

C) npx prisma studio

D) npx tsc --init

E) npm install prisma

17. No TypeScript, qual a função do pacote ts-node instalado nas dependências de desenvolvimento?

A) Compilar o código TS permanentemente para JS em uma pasta de produção.

B) Criar automaticamente as tabelas no banco de dados relacional.

C) Executar arquivos .ts diretamente no Node.js, agilizando o desenvolvimento.

D) Gerar o arquivo de configuração tsconfig.json.

E) Gerenciar as variáveis de ambiente do arquivo .env.

18. No Prisma, o que o ponto de interrogação (?) representa em uma definição de campo como description String??

A) O campo é uma chave primária.

B) O campo é obrigatório e não pode ser nulo.

C) O campo é opcional (pode conter valor nulo).

D) O campo será preenchido automaticamente pelo banco.

E) O campo é uma busca (query).

19. Qual arquivo é responsável por armazenar a DATABASE_URL, garantindo que informações sensíveis não fiquem expostas no código principal?

A) schema.prisma

B) package.json

C) tsconfig.json

D) .env

E) index.ts

20. Considere o seguinte erro no console ao tentar rodar a aplicação:

```
PrismaClientKnownRequestError: P2002: Unique constraint failed on the fields:
 (email)
```

21. O que este erro indica sobre a operação que o código tentou realizar?

A) O banco de dados MongoDB está offline.

B) O campo "email" não existe no modelo do Prisma.

C) Tentou-se inserir um email que já existe em um campo marcado com @unique.

D) A conexão com a internet falhou durante a migração.

E) O tipo de dado enviado para o email não é uma String.

22. Analise o trecho de código TypeScript abaixo:

```
const result = await prisma.user.findMany({
  where: {
    active: true
  },
  take: 5
});
```

O que a variável result retornará?

A) Todos os usuários ativos do banco de dados.

B) Apenas o primeiro usuário que encontrar com o status ativo.

C) Uma lista com, no máximo, 5 usuários que possuem active como true.

D) Um erro, pois o método take não existe no Prisma.

E) O número total (count) de usuários ativos.

23. Qual a principal diferença entre o comando npx prisma migrate dev e o npx prisma db push?

A) O migrate dev é exclusivo para SQL e o db push é a forma recomendada para bancos NoSQL como MongoDB.

B) O db push apaga o banco de dados e o migrate dev apenas atualiza.

C) O migrate dev não gera histórico de alterações, enquanto o db push gera.

D) O db push serve apenas para baixar os dados do banco para o código.

E) Não há diferença; os dois comandos fazem exatamente a mesma coisa.

24. Observe a configuração do tsconfig.json abaixo:

```
{
  "compilerOptions": {
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true
  }
}
```

Ao rodar o comando npx tsc, onde os arquivos JavaScript resultantes serão salvos?

A) Na raiz do projeto.

B) Dentro da pasta src. C) Dentro da pasta dist.

D) Dentro da pasta node_modules.

E) Os arquivos não serão salvos, apenas executados na memória.

25. Explique o conceito de "Introspection" (Engenharia Reversa) no Prisma e em que cenário um desenvolvedor usaria essa funcionalidade.

26. Estudo de Caso:

Você está criando um sistema de "Lista de Tarefas". Cada Tarefa pertence a uma Categoria (ex: Trabalho, Lazer). Escreva o trecho do schema.prisma que represente essa relação, garantindo que, ao deletar uma Categoria, as Tarefas vinculadas a ela também sejam tratadas (onDelete).

26. Estudo de Caso: Relacionamentos

Você tem um modelo de Author e um modelo de Book. No Prisma, você define:

```
model Author {
  id    String @id @default(auto()) @map("_id") @db.ObjectId
  books Book[]
}

model Book {
  id      String @id @default(auto()) @map("_id") @db.ObjectId
  title    String
  author    Author @relation(fields: [authorId], references: [id])
  authorId String @db.ObjectId
}
```

Se você quiser buscar um autor e já trazer todos os livros dele em uma única consulta no TypeScript, qual sintaxe deve usar?

A) `prisma.author.findMany({ where: { books: true } })`

B) `prisma.author.findUnique({ where: { id }, include: { books: true } })`

C) `prisma.book.findMany({ author: true })`

D) `prisma.author.findUnique({ where: { id }, join: "books" })`

E) `prisma.author.findUnique({ id }).books()`

27. Implementação de Middleware/Lógica: Como você garantiria, usando TypeScript, que o campo quantity de um item nunca seja salvo com um valor negativo antes de enviar o comando prisma.create()? Escreva um pequeno trecho de lógica de validação.

28. Estudo de Caso: Transações Imagine que você precisa transferir um item de uma "Lista A" para uma "Lista B". Isso envolve deletar de uma e criar na outra. Se a criação na "Lista B" falhar, o item não pode sumir da "Lista A". Como o comando prisma.$transaction([]) resolve esse problema?

29. Ao usar o Prisma, às vezes precisamos de um tipo que represente o modelo, mas com os relacionamentos incluídos. Como o Prisma gera esses tipos automaticamente para usarmos em funções TypeScript?

30. Mencionamos através das aulas que o Prisma é "Type-safe". Explique como essa característica diminui a incidência de bugs em grandes aplicações em comparação ao uso de drivers nativos (como o mongodb driver puro).

31. Analise o modelo abaixo e responda: Como o campo id se comportará no MongoDB?

```
model Post {
  id    String @id @default(auto()) @map("_id") @db.ObjectId
  title String
}
```

A) Será um número inteiro autoincrementado (1, 2, 3...).

B) Será um campo comum que o usuário deve preencher manualmente.

C) Será o ID padrão do MongoDB (_id), gerado automaticamente como um ObjectId.

D) O Prisma dará erro porque o MongoDB não aceita o decorador @map.

32. Considere o seguinte trecho de código TypeScript:

```
const newItem = await prisma.shoppingItem.create({
  data: {
    name: "Leite",
    quantity: 2,
    isDone: false
  }
})
```

O que a variável newItem conterá após a execução?

A) Apenas uma mensagem de "Sucesso".

B) Um array com todos os itens do banco de dados.

C) Um objeto representando o item recém-criado, incluindo o id gerado pelo banco.
D) O código SQL gerado pelo Prisma.

33. Complete o código: Você precisa buscar todos os itens da lista de compras que já foram finalizados (isDone igual a true).

```
const items = await prisma.shoppingItem.findMany({
  where: {
    // PREENCHA AQUI
  }
})
```

34. No arquivo schema.prisma, como você define que um campo email deve ser único (não pode haver dois iguais)?

A) `email String @primary`
B) `email String @unique`
C) `email String @id`
D) `email String @default(unique)`

35. Estudo de Caso: Você está recebendo o erro `"Cannot find module '@prisma/client'"` ao tentar rodar seu projeto. Quais são os dois passos necessários para resolver isso?